

Domain and Problem Statement

Domain: Financial Analytics & Customer Transaction Analysis

Problem Statement: Financial transaction systems generate large volumes of customer, transaction, credit, income, debt and merchant-related data. Manual analysis makes it difficult to identify spending patterns, compare customer profiles, evaluate credit characteristics and understand debt-related behaviour. This project transforms the raw financial dataset into an analytical solution using Python, MySQL, Power BI and Tableau.

Dataset Description

Dataset Name: Customer Financial Profiles

Dataset Size: 20,000 records | Original Columns: 21 | Final Analytical Columns: 31

Granularity: Each row represents a financial transaction record containing customer profile information, transaction details, credit characteristics, payment information and merchant information.

Key Entities & Attributes

- Customer Information – CustomerID, ClientID, CurrentAge, BirthYear, BirthMonth, Gender and Address.
- Financial Profile – PerCapitaIncome, YearlyIncome and TotalDebt.
- Credit Information – CreditScore, NumCreditCards and CreditScoreCategory.
- Transaction Information – TransactionID, Date, Amount and TransactionAmountCategory.
- Merchant Information – MerchantID, MerchantCity, MerchantState and Zip.
- Analytical Indicators – DebtToIncomeRatio, AgeGroup, IncomeCategory and DebtStatus.
- Time Attributes – TransactionYear, TransactionMonth, TransactionMonthName and TransactionQuarter.

Data Source

Primary Source: Customer Financial Profiles dataset. Processing: Python / Pandas and NumPy.

Database: MySQL 8.0 – financial_analytics. Visualization: Microsoft Power BI and Tableau Public.

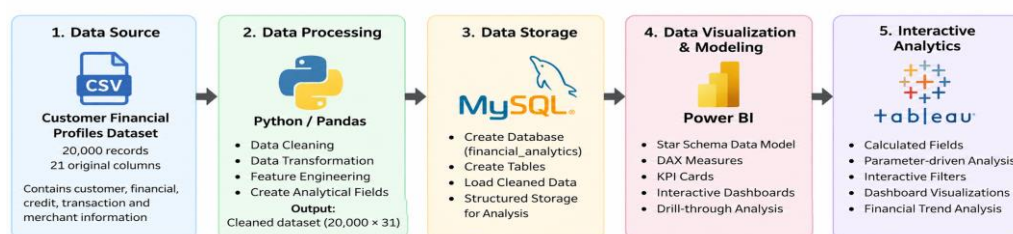


Fig 1: Financial Customer & Transaction Analytics Project Architecture

1. Python – Data Cleaning & Transformation

Loading the Dataset

```
import pandas as pd
import numpy as np

df = pd.read_csv("../financial/Customer_Financial_Profiles.csv")
print("Original Shape:", df.shape)
```

Standardizing Column Names

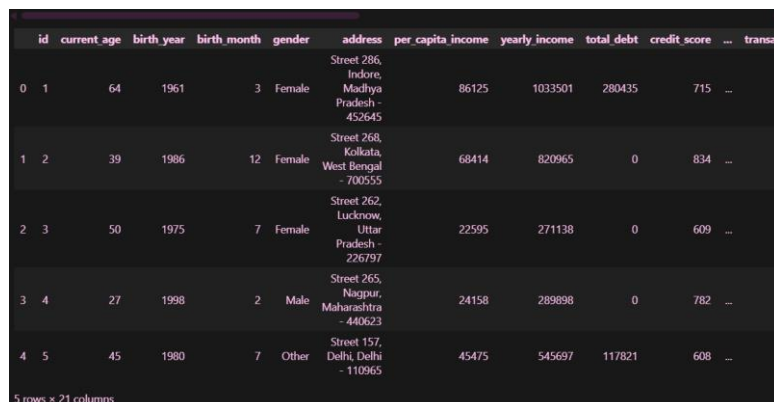
```
df.columns = [
    "CustomerID", "CurrentAge", "BirthYear", "BirthMonth",
    "Gender", "Address", "PerCapitaIncome", "YearlyIncome",
    "TotalDebt", "CreditScore", "NumCreditCards",
    "TransactionID", "Date", "ClientID", "CardID", "Amount",
    "UseChip", "MerchantID", "MerchantCity", "MerchantState", "Zip"
]
```

Data Type Conversion

```
numeric_cols = [
    "CustomerID", "CurrentAge", "BirthYear", "BirthMonth",
    "PerCapitaIncome", "YearlyIncome", "TotalDebt",
    "CreditScore", "NumCreditCards", "Amount"
]

for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors="coerce")

df["Date"] = pd.to_datetime(df["Date"], errors="coerce")
```



	id	current_age	birth_year	birth_month	gender	address	per_capita_income	yearly_income	total_debt	credit_score	...	transac
0	1	64	1961	3	Female	Street 286, Indore, Madhya Pradesh - 452645	86125	1033501	280435	715	...	...
1	2	39	1986	12	Female	Street 268, Kolkata, West Bengal - 700555	68414	820965	0	834	...	...
2	3	50	1975	7	Female	Street 262, Lucknow, Uttar Pradesh - 226797	22595	271138	0	609	...	...
3	4	27	1998	2	Male	Street 265, Nagpur, Maharashtra - 440623	24158	289898	0	782	...	...
4	5	45	1980	7	Other	Street 157, Delhi, Delhi - 110965	45475	545697	117821	608	...	...

5 rows × 21 columns

Fig 2: Raw Financial Dataset Loaded in VS Code

Python – Validation & Feature Engineering

Duplicate Handling

```
duplicates_before = df.duplicated().sum()
df = df.drop_duplicates()
duplicates_after = df.duplicated().sum()
```

Debt-to-Income Ratio

```
df["DebtToIncomeRatio"] = np.where(
    df["YearlyIncome"] > 0,
    (df["TotalDebt"] / df["YearlyIncome"]) * 100,
    0
)
```

Age and Credit Segmentation

```
df["AgeGroup"] = pd.cut(
    df["CurrentAge"],
    bins=[18, 25, 35, 45, 55, 65, 100],
    labels=["19-25", "26-35", "36-45", "46-55", "56-65", "66+"]
)

df["CreditScoreCategory"] = pd.cut(
    df["CreditScore"],
    bins=[0, 579, 669, 739, 799, 900],
    labels=["Poor", "Fair", "Good", "Very Good", "Excellent"]
)
```

Transaction Amount and Time Features

```
df["TransactionAmountCategory"] = pd.cut(
    df["Amount"],
    bins=[0, 1000, 5000, 10000, np.inf],
    labels=["Low", "Medium", "High", "Very High"]
)
df["TransactionYear"] = df["Date"].dt.year
df["TransactionMonth"] = df["Date"].dt.month
df["TransactionMonthName"] = df["Date"].dt.month_name()
df["TransactionQuarter"] = "Q" + df["Date"].dt.quarter.astype(str)
```

The engineered fields convert raw numerical and date attributes into meaningful analytical categories for Power BI and Tableau.

Python – Final Validation and Dataset Export

Validation

```
print("=== CLEANED DATASET ===")
print("Shape:", df.shape)
print("\nMissing Values:")
print(df.isnull().sum())
print("\nDuplicate Rows:")
print(df.duplicated().sum())
print("\nCredit Score Range:")
print(df["CreditScore"].min(), "to", df["CreditScore"].max())
print("\nAge Range:")
print(df["CurrentAge"].min(), "to", df["CurrentAge"].max())
print("\nTransaction Amount Range:")
print(df["Amount"].min(), "to", df["Amount"].max())
print("\nDebt-to-Income Ratio Range:")
print(round(df["DebtToIncomeRatio"].min(), 2),
      "to", round(df["DebtToIncomeRatio"].max(), 2))
```

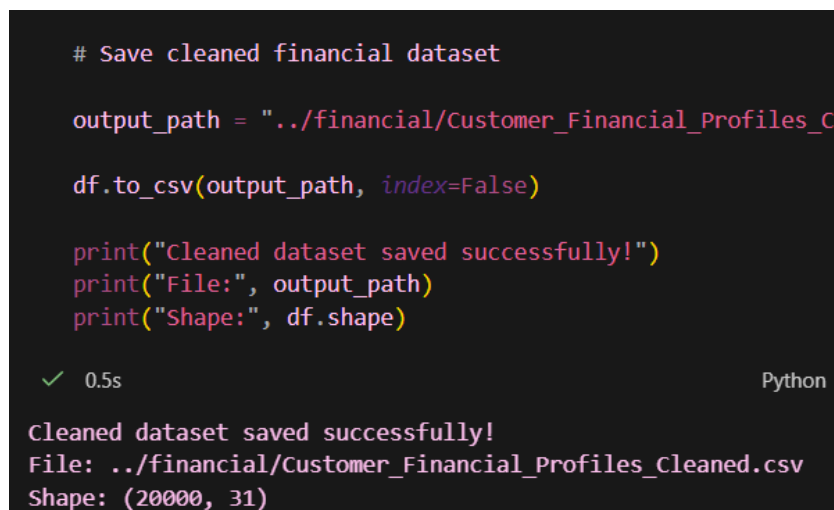
Final Dataset Export

```
output_path = "../financial/Customer_Financial_Profiles_Cleaned.csv"

df.to_csv(output_path, index=False)

print("Cleaned dataset saved successfully!")
print("File:", output_path)
print("Shape:", df.shape)
```

Final cleaned dataset: 20,000 rows × 31 analytical columns.



```
# Save cleaned financial dataset

output_path = "../financial/Customer_Financial_Profiles_C

df.to_csv(output_path, index=False)

print("Cleaned dataset saved successfully!")
print("File:", output_path)
print("Shape:", df.shape)
```

✓ 0.5s Python

Cleaned dataset saved successfully!
File: ../financial/Customer_Financial_Profiles_Cleaned.csv
Shape: (20000, 31)

Fig 3: Cleaned Dataset Export and Final Shape

2. MySQL – Database and Data Loading

Database Creation

```
CREATE DATABASE financial_analytics;  
USE financial_analytics;
```

Financial Transaction Table

```
CREATE TABLE customer_financial_profiles (  
    CustomerID INT,  
    CurrentAge INT,  
    BirthYear INT,  
    BirthMonth INT,  
    Gender VARCHAR(20),  
    Address VARCHAR(255),  
    PerCapitaIncome DECIMAL(12,2),  
    YearlyIncome DECIMAL(14,2),  
    TotalDebt DECIMAL(14,2),  
    CreditScore INT,  
    NumCreditCards INT,  
    TransactionID VARCHAR(30),  
    Date DATE,  
    ClientID VARCHAR(30),  
    CardID VARCHAR(30),  
    Amount DECIMAL(12,2),  
    UseChip VARCHAR(10),  
    MerchantID VARCHAR(30),  
    MerchantCity VARCHAR(100),  
    MerchantState VARCHAR(100),  
    Zip VARCHAR(10),  
    DebtToIncomeRatio DECIMAL(10,2),  
    AgeGroup VARCHAR(30),  
    CreditScoreCategory VARCHAR(30),  
    IncomeCategory VARCHAR(40),  
    DebtStatus VARCHAR(20),  
    TransactionAmountCategory VARCHAR(20),  
    TransactionYear INT,  
    TransactionMonth INT,  
    TransactionMonthName VARCHAR(20),  
    TransactionQuarter VARCHAR(5)  
);
```

Python–MySQL Connection

```
import mysql.connector

conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="YOUR_MYSQL_PASSWORD",
    database="financial_analytics"
)

cursor = conn.cursor()
print("MySQL connection successful!")
```

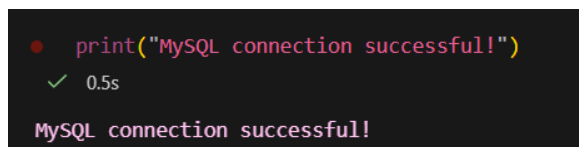
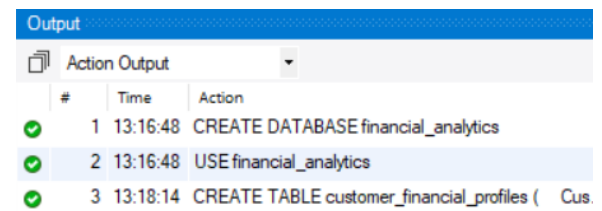


Fig 4: Successful Python–MySQL Connection

A screenshot of the 'Output' window in a MySQL IDE. The 'Action Output' tab is selected. It displays a table with three columns: '#', 'Time', and 'Action'. There are three rows of data, each preceded by a green checkmark icon.

#	Time	Action
1	13:16:48	CREATE DATABASE financial_analytics
2	13:16:48	USE financial_analytics
3	13:18:14	CREATE TABLE customer_financial_profiles (Cus.

Fig 5: MySQL Database Action Output

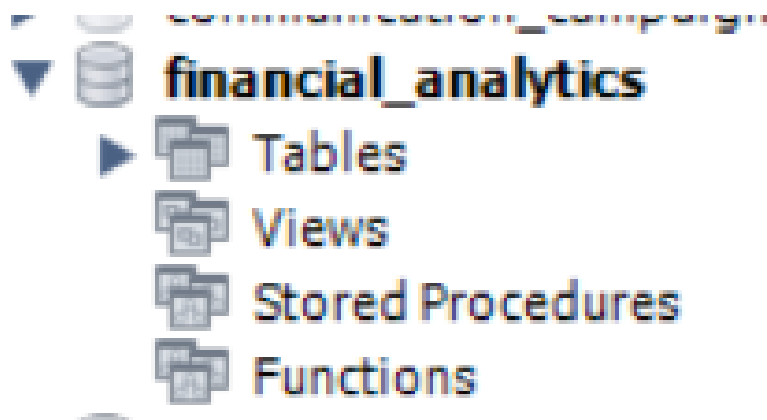


Fig 6: MySQL financial_analytics schema/table view

Procedure

Step 1: Data Preparation & Preprocessing

- Load the Customer Financial Profiles dataset using Pandas.
- Inspect the dataset structure, data types, missing values and duplicate records.
- Standardize column names and convert numerical and date fields.
- Clean identifier and categorical fields.

Step 2: Feature Engineering

- Calculate Debt-to-Income Ratio.
- Create Age Groups and Credit Score Categories.
- Create Income Categories and Debt Status.
- Categorize transaction amounts and generate transaction time fields.

Step 3: SQL Database Connection & Data Ingestion

- Create the financial_analytics database in MySQL 8.0.
- Create customer_financial_profiles with 31 analytical columns.
- Load the cleaned dataset using Python MySQL Connector in batches.
- Verify the final record count in MySQL.

Step 4: Power BI Analytics

- Connect Power BI Desktop to the local MySQL database.
- Create a star-schema-style model using fact and dimension tables.
- Develop DAX measures and KPI cards.
- Create interactive dashboards, decomposition analysis and Key Influencers.
- Implement customer-level drill-through.

Step 5: Tableau Analytics

- Connect Tableau Public to the cleaned CSV.
- Create analytical worksheets for financial, credit, merchant and payment analysis.
- Create calculated fields and the Select Financial Metric parameter.
- Apply filters and dashboard interactions.

3. Power BI – Data Connection & Preparation

Connecting Power BI to MySQL

- Open Power BI Desktop and select Home → Get Data → MySQL Database.
- Server: localhost.
- Database: financial_analytics.
- Authenticate using the local MySQL credentials.
- Select customer_financial_profiles and load the table.

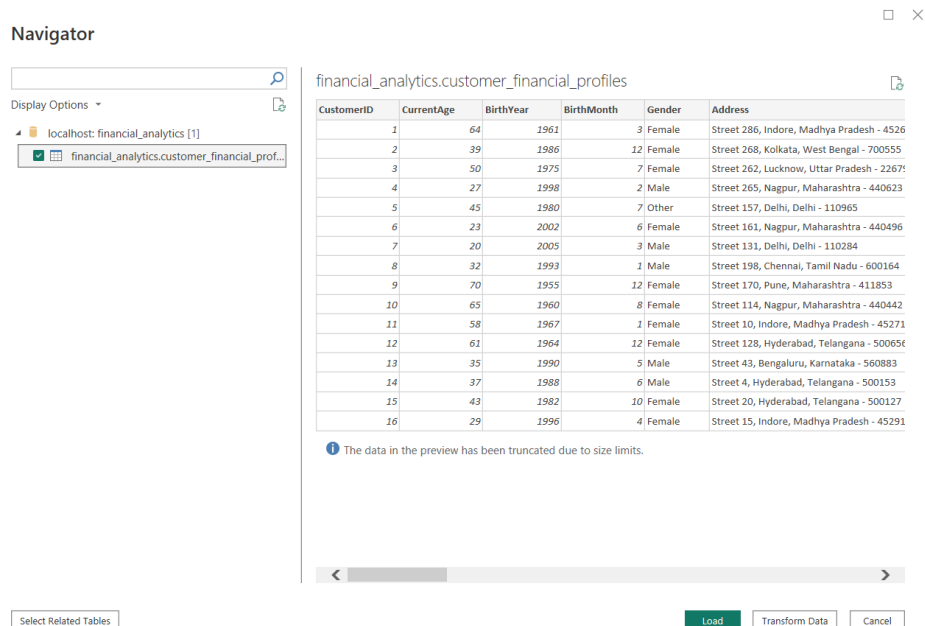


Fig 7: Power BI Navigator – financial_analytics.customer_financial_profiles

Imported Power BI Table

The imported table was renamed to FactFinancialTransactions to represent the central transaction-level fact table. The data contains customer profiles, financial attributes, transaction details, merchant information and engineered analytical fields.

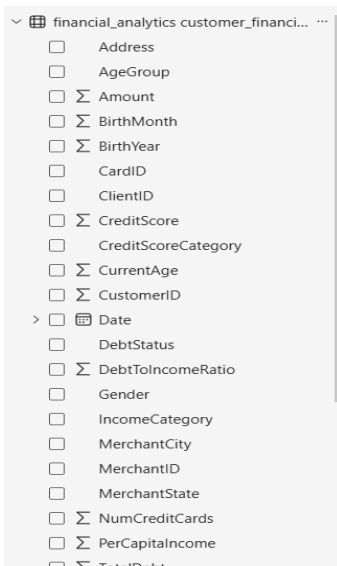


Fig 8: Financial fields available in Power BI

Step 3: Star Schema Data Modeling

Fact Table

FactFinancialTransactions contains the transaction-level records and the associated customer, financial, credit, payment and merchant attributes.

Dimension Tables

- DimCustomer – ClientID and customer demographic/financial profile attributes.
- DimMerchant – unique MerchantID values used to relate merchant information to transactions.
- DimPayment – UseChip payment classification.
- DimDate – calendar dates with Year, Month, Quarter and Year-Month attributes.

Relationships

The model uses one-to-many relationships from each dimension to FactFinancialTransactions with single-direction filtering. This structure supports reusable dimensions and interactive slicing of transaction data.

```
1 DimMerchant =  
2 DISTINCT  
3 SELECTCOLUMNS(  
4     FactFinancialTransactions,  
5     "MerchantID", FactFinancialTransactions[MerchantID]  
6 )  
7 ]
```

Fig 9: DimMerchant DAX Table

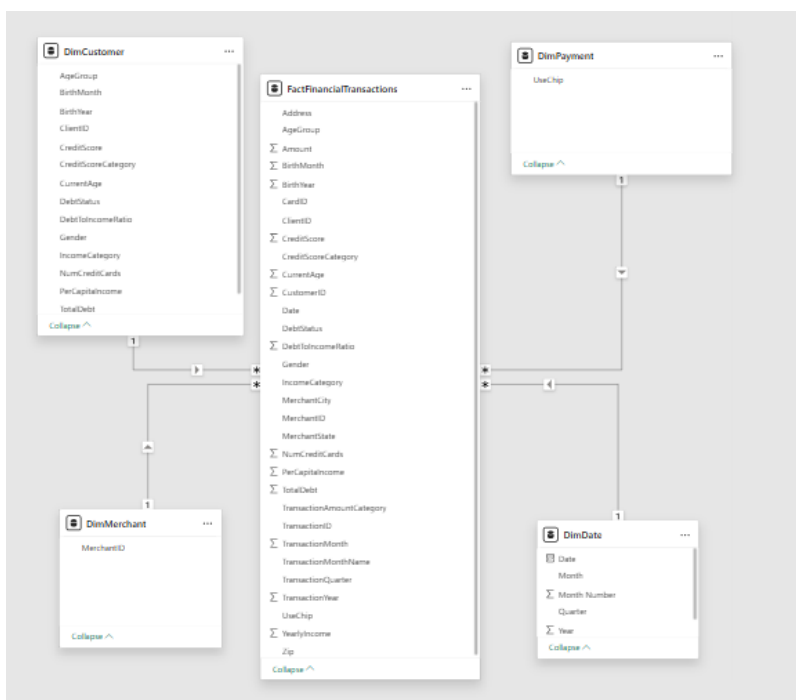


Fig 10: Power BI Star-Schema-Style Financial Data Model

Step 4: Core Calculations & Measures (DAX)

Transaction Measures

```
Total Transactions =  
DISTINCTCOUNT(FactFinancialTransactions[TransactionID])  
Total Customers =  
DISTINCTCOUNT(FactFinancialTransactions[ClientID])  
Total Transaction Amount =  
SUM(FactFinancialTransactions[Amount])  
Average Transaction Amount =  
AVERAGE(FactFinancialTransactions[Amount])
```

Customer-Level Financial Measures

```
Average Credit Score =  
AVERAGEX(  
    VALUES(FactFinancialTransactions[ClientID]),  
    CALCULATE(MAX(FactFinancialTransactions[CreditScore])))  
  
Average Yearly Income =  
AVERAGEX(  
    VALUES(FactFinancialTransactions[ClientID]),  
    CALCULATE(MAX(FactFinancialTransactions[YearlyIncome])))  
  
Total Customer Debt =  
SUMX(  
    VALUES(FactFinancialTransactions[ClientID]),  
    CALCULATE(MAX(FactFinancialTransactions[TotalDebt])))
```

```
1 Total Transactions =  
2 DISTINCTCOUNT(  
3     FactFinancialTransactions[TransactionID]  
4 )
```

Fig 11: Total Transactions Measure

```
✗ ✓ 1 Total Customers =  
2 DISTINCTCOUNT(  
3     FactFinancialTransactions[ClientID]  
4 )
```

Fig 12: Total Customers Measure

```
1 Total Transaction Amount =  
2 SUM(  
3     FactFinancialTransactions[Amount]  
4 )
```

Fig 13 & 14: Total Transaction Amount Measure
&
Power BI Financial Tables

Data

Q Search

- > DimCustomer
- > DimDate
- > DimMerchant
- > DimPayment
- > FactFinancialTransactions

Step 5: Power BI – Customer Financial Analytics Dashboard

The main Power BI dashboard combines customer KPIs, transaction trends, merchant-state comparison, credit-score distribution, age-group analysis, payment behaviour and interactive decomposition analysis.

KPI Cards

Total Customers: 4,941 | Total Transactions: 13K | Total Transaction Amount: ₹61.65M | Average Credit Score: 702 | Debt Customer Rate: 64.2%

Dashboard Visualizations

- Transaction Amount Trend – tracks transaction amount over Year-Month.
- Transactions by Merchant State – compares transaction activity across states.
- Customers by Credit Score Category – segments customers by credit profile.
- Customers by Age Group – compares the customer population by age band.
- Transaction Method – compares Yes/No UseChip transaction counts.
- Transaction Amount Decomposition – interactively breaks down transaction amount by analytical dimensions.

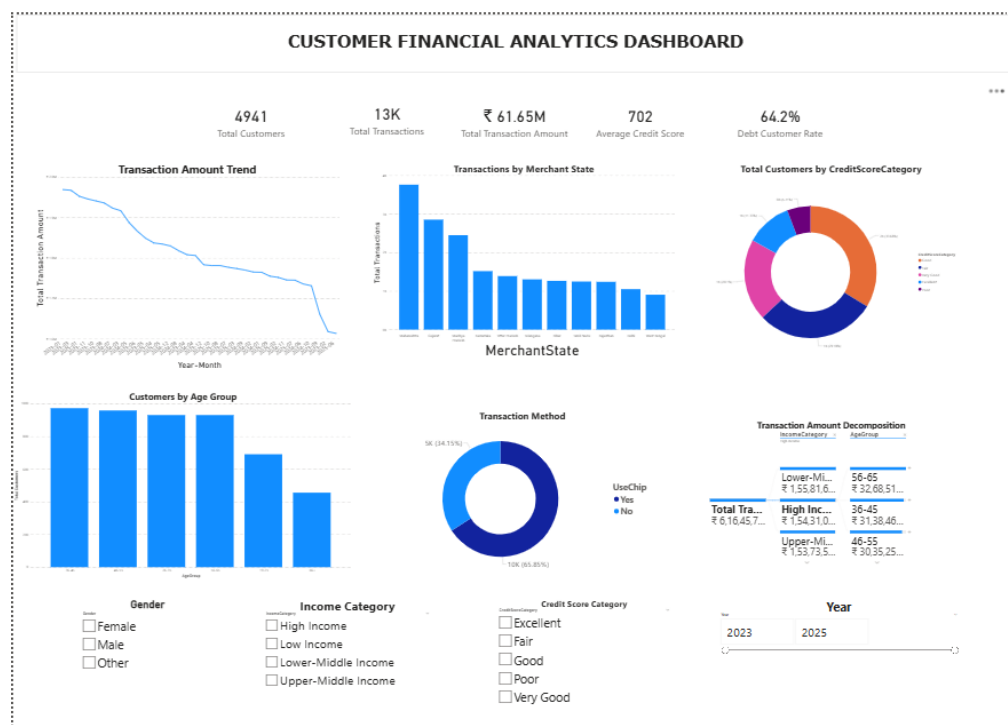


Fig 15: Power BI Customer Financial Analytics Dashboard

Power BI – Customer & Financial Risk Analysis

A dedicated analytical page was created to study relationships between income, debt, credit score and customer financial characteristics.

Analytical Visuals

- Income vs Total Debt by Credit Score – scatter analysis of yearly income and total debt with credit-score categories.
- Credit Score vs Transaction Amount – examines transaction amount distribution across credit scores and transaction categories.
- Total Debt by Income Category – compares customer debt across income segments.
- Debt-to-Income Ratio by Age Group – compares average debt-to-income ratios across age groups.
- Key Influencers – provides interactive analysis of factors associated with transaction amount.

The page complements the executive dashboard by focusing on financial risk, customer segmentation and relationships among financial variables.



Fig 16: Power BI Customer & Financial Risk Analysis

Power BI – Customer-Level Drill-Through

Drill-Through Configuration

A Customer Details page was configured using ClientID as the drill-through field. When a customer is selected from a report visual, Power BI passes the ClientID context to the detailed page.

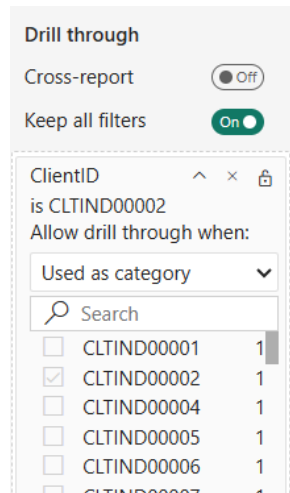


Fig 17: ClientID Drill-Through Configuration

Customer Details Page

- Customer profile including ClientID, Gender, Age Group, Income Category, Credit Score Category, Debt Status and Number of Credit Cards.
- Financial information including Yearly Income, Total Debt and Debt-to-Income Ratio.
- Transaction History containing Date, TransactionID, Amount, UseChip, MerchantCity, MerchantState and TransactionAmountCategory.
- Back navigation is provided to return to the source dashboard.

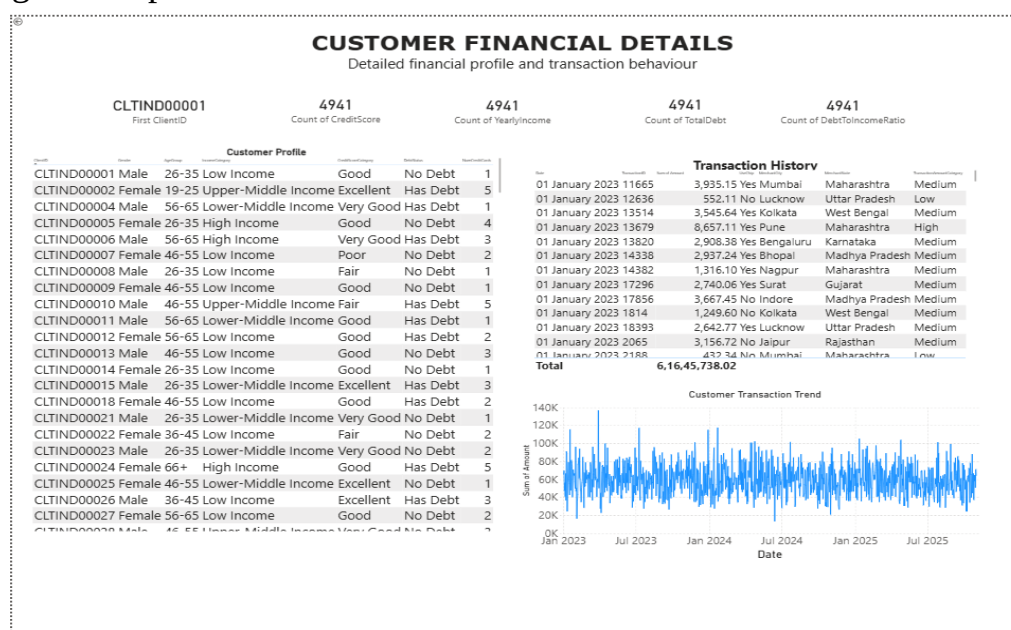


Fig 18: Customer Financial Details Drill-Through Page

4. Tableau – Data Connection & Preparation

Connecting the Cleaned Dataset

- Open Tableau Public.
- Select Connect → Text File.
- Load Customer_Financial_Profiles_Cleaned.csv.
- Verify that the data source contains 31 fields and 20,000 rows.

Customer_Financial_Profiles_Cleaned

Filters
0 | Add

Customer_Financial_Pro...

Customer_Financial_Profil... 31 fields 20000 rows 100 rows

Name Customer_Financial_Profiles_Cleaned.csv

Description
No description available.

Fields

Type	Field Name	Physical Table	Rem...
#	Customer ID	Customer_Financi...	Custo...
#	Current Age	Customer_Financi...	Curre...
#	Birth Year	Customer_Financi...	BirthY...
#	Birth Month	Customer_Financi...	Birth...

#	Customer ID	Current Age	Birth Year	Birth Month
1		64	1961	3
2		39	1986	12
3		50	1975	7
4		27	1998	2
5		45	1980	7
6		23	2002	6
7		20	2005	3
8		32	1993	1
9		70	1955	12

Fig 19 : Tableau Data Source – Customer_Financial_Profiles_Cleaned

Prepared Analytical Fields

The cleaned CSV provides customer, financial, credit, transaction, payment, merchant and time attributes for Tableau analysis. Tableau was used with the cleaned CSV for the visualization stage, while MySQL served as the centralized database layer for the Power BI workflow.

Tableau – Core Worksheets

Financial Metric Analysis

A parameter-driven line chart was created to compare a selected financial metric across transaction years.

Transactions by Merchant State

A bar chart uses distinct TransactionID counts to compare transaction activity across merchant states.

Customers by Credit Score Category

A bar chart compares distinct customers across Excellent, Fair, Good, Poor and Very Good credit-score categories.

Income vs Debt by Credit Score

A scatter plot compares Yearly Income and Total Debt while using Credit Score Category to distinguish customer profiles.

Transactions by Payment Method

A bar chart compares distinct transactions for UseChip = Yes and No.

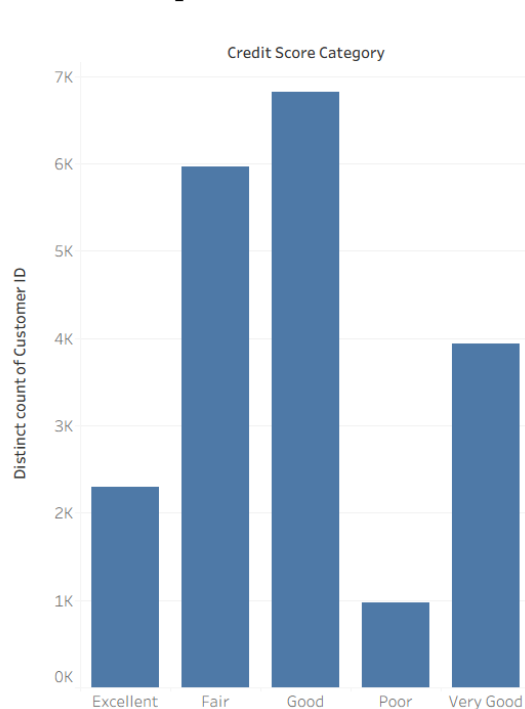


Fig 20: Credit Score Category

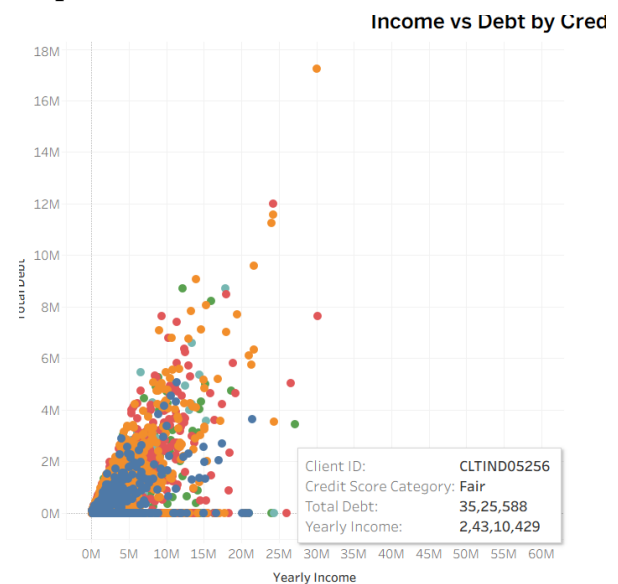


Fig 21: Income vs Debt by Credit Score

Transactions by Merchant State

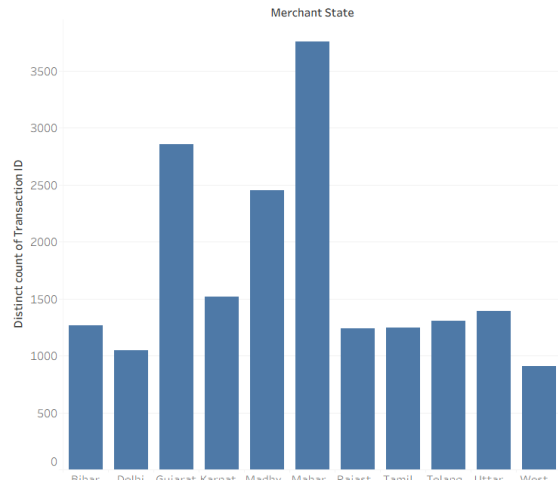


Fig 22: Transactions by Merchant State

Transactions by Payment Method

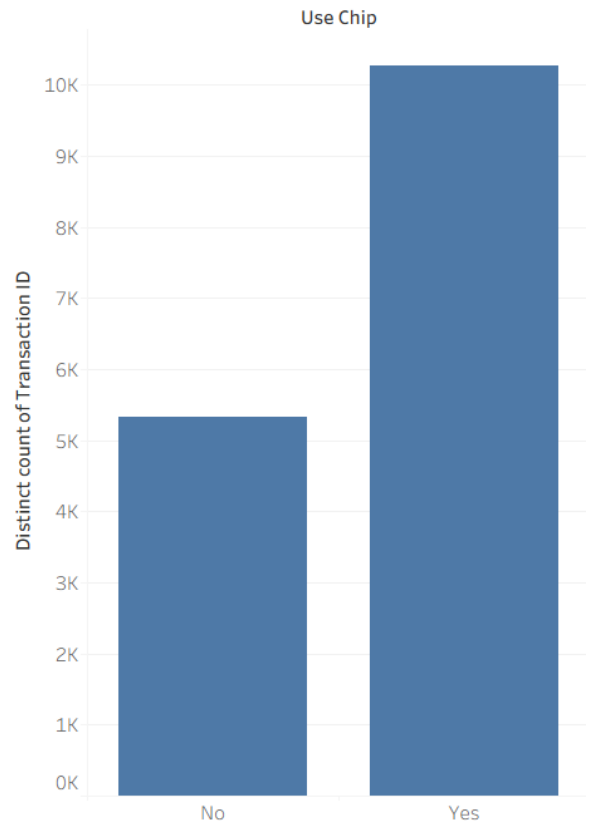


Fig 23: Transactions by Payment Method

Tableau – Calculated Fields & Parameter

Calculated Field: Transaction Amount

```
SUM([Amount])
```

Calculated Field: Debt to Income %

Debt to Income %

```
AVG([DebtToIncomeRatio])
```

Fig 24: Tableau Debt to Income % Calculated Field

Create Parameter – Select Financial Metric

Data Type: String | Current Value: Transaction Amount | Allowable Values: Transaction Amount, Total Debt, Yearly Income, Credit Score

Create Parameter

Name
Select Financial Metric

Properties

Data type: String
Display format: Transaction Amount

Current value: Transaction Amount
Value when workbook opens: Current value

Allowable values
☐ All ☒ List ☐ Range

Value	Display As
Transaction Amount	Transaction Amount
Total Debt	Total Debt
Yearly Income	Yearly Income
Credit Score	Credit Score
Click to add	

☒ Fixed
☐ When workbook opens

Add values from

Remove Selected

Cancel OK

Fig 25: Tableau Select Financial Metric Parameter

Selected Financial Metric

```
CASE [Select Financial Metric]
WHEN "Transaction Amount" THEN SUM([Amount])
WHEN "Total Debt" THEN SUM([TotalDebt])
WHEN "Yearly Income" THEN SUM([YearlyIncome])
WHEN "Credit Score" THEN AVG([CreditScore])
END
```

Tableau – Parameter-Driven Analysis & Filters

The Select Financial Metric parameter controls the Selected Financial Metric calculated field. The line chart changes according to the selected financial measure.

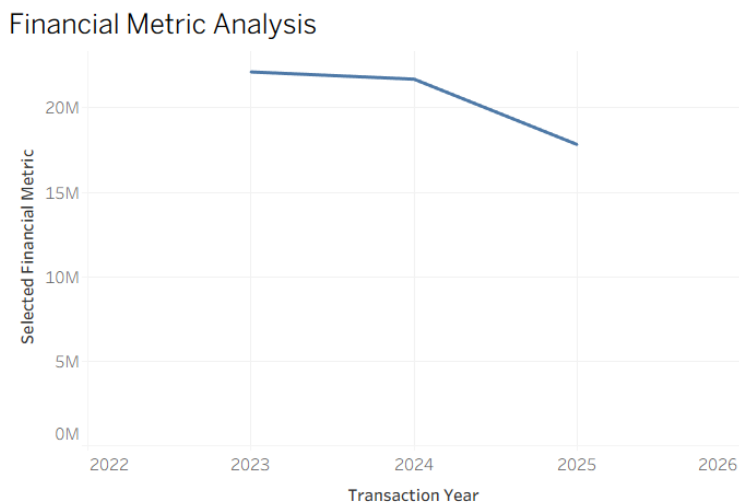


Fig 26: Tableau Financial Metric Analysis

Interactive Filters

Dashboard filters were created for Gender, Income Category and Credit Score Category. These filters allow users to focus the analysis on selected customer segments.

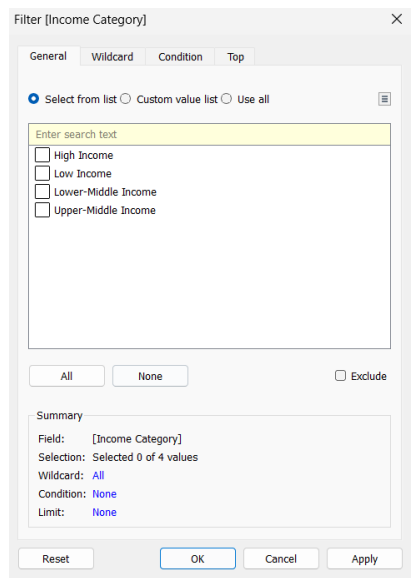


Fig 27: Tableau Income Category Filter

Dashboard Interaction

The financial dashboard supports interactive filtering and selection-based exploration. A state selected in the Transactions by Merchant State view can be used to filter the other dashboard views.

Tableau – Final Financial Analytics Dashboard

The Tableau dashboard combines the parameter-driven Financial Metric Analysis, Income vs Debt by Credit Score, Customer Credit Score Distribution, Transactions by Merchant State and Transactions by Payment Method.

Dashboard Components

- Financial Metric Analysis – dynamic metric selection through a Tableau parameter.
- Income vs Debt by Credit Score – relationship between customer income and debt.
- Customers by Credit Score Category – customer segmentation.
- Transactions by Merchant State – geographical transaction comparison.
- Transactions by Payment Method – payment behaviour comparison.
- Filters – Gender, Income Category and Credit Score Category.

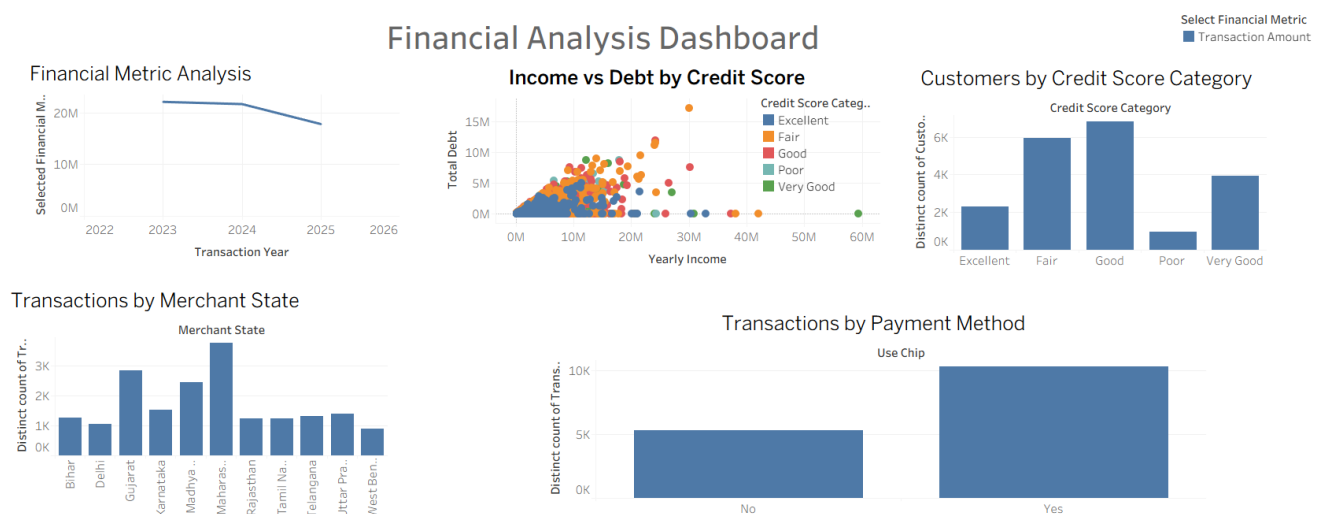


Fig 28: Tableau Financial Analysis Dashboard

Observed Insights

- The project processes 20,000 financial records and produces 31 analytical fields after cleaning and feature engineering.
- There are 4,941 unique ClientIDs represented in the dataset, while the transaction analysis identifies approximately 13,028 distinct TransactionIDs.
- The Power BI executive dashboard reports a total transaction amount of approximately ₹61.65 million.
- The average customer credit score shown on the Power BI dashboard is approximately 702.
- The dashboard indicates a 64.2% customer debt rate, showing that a substantial portion of customers have outstanding debt.
- Merchant-state transaction volumes differ considerably, enabling geographical comparison of transaction activity.
- The credit-score distribution shows the largest customer groups in the Good and Fair categories, while Poor and Excellent categories are smaller.
- Chip-enabled transactions are more frequent than non-chip transactions in the Tableau payment-method analysis.
- The income-versus-debt scatter analysis shows a positive relationship between yearly income and total debt, with variation across credit-score categories.
- The Debt-to-Income Ratio analysis allows financial burden to be compared across age groups.
- Power BI decomposition and Key Influencers provide interactive exploration beyond static charts.
- Customer drill-through provides transaction-level detail for an individual ClientID.

Conclusion

The project demonstrates an end-to-end financial analytics workflow in which Python performs data cleaning and feature engineering, MySQL provides structured storage, Power BI provides interactive KPI, risk and drill-through analysis, and Tableau Public provides complementary parameter-driven visual analytics. The resulting dashboards support customer segmentation, transaction analysis, credit-score evaluation, income and debt analysis, merchant-state comparison and payment-method analysis.

Project Deliverables

Raw Dataset | Cleaned CSV | Python Notebook | MySQL Database | Power BI Dashboard (.pbix) | Tableau Workbook/Dashboard | DAX Measures | Calculated Fields | Parameter | Filters | Drill-Through | Documentation

Technologies Used

Python | Pandas | NumPy | MySQL 8.0 | Power BI | DAX | Tableau Public